

Week 3 Portfolio Milestone in C

John Wensink

CSC507 Foundations of Operating Systems

Colorado State University – Global Campus

Dr. Joseph Issa

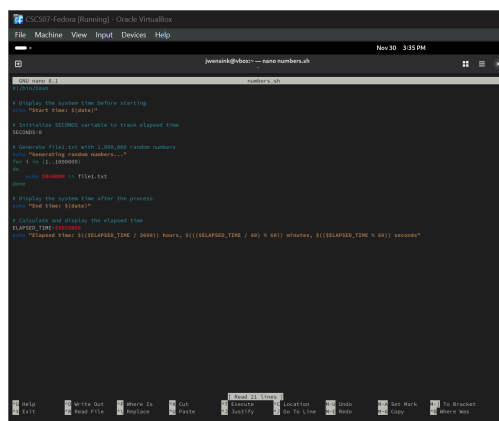
November, 30th, 2024

Week 3 Portfolio Milestone in C

For this week's Portfolio Milestone project, I started adding improvements to my shell script and C program. The first objective was to display the system time before and after the process, which was accomplished first in the shell script. The top of the script contains a shebang/hashbang **#!/bin/bash** to mark the file as a script and specifies **bash** as the interpreter that will be used. Using the **date** command, I captured and output the start and end times of the script's execution. After this was shown to work with basic functionality, I added the variable **\$ELAPSED_TIME** to store the total time in seconds after the script started running. To output the time in a more readable format, the command

echo "Elapsed time: \$(((\$ELAPSED_TIME / 3600)) hours, \$(((\$ELAPSED_TIME / 60) % 60)) minutes, \$(((\$ELAPSED_TIME % 60)) seconds)"

converts the unwieldy total number of seconds into hours, minutes, and seconds. The shell script took a surprising 1 minute and 36 seconds to create file1.txt with 1 million lines of random numbers.



```
#!/bin/bash

# Display the system time before starting
echo "Start time: $(date)"

# Initialize SECONDS variable to track elapsed time
SECONDS=0

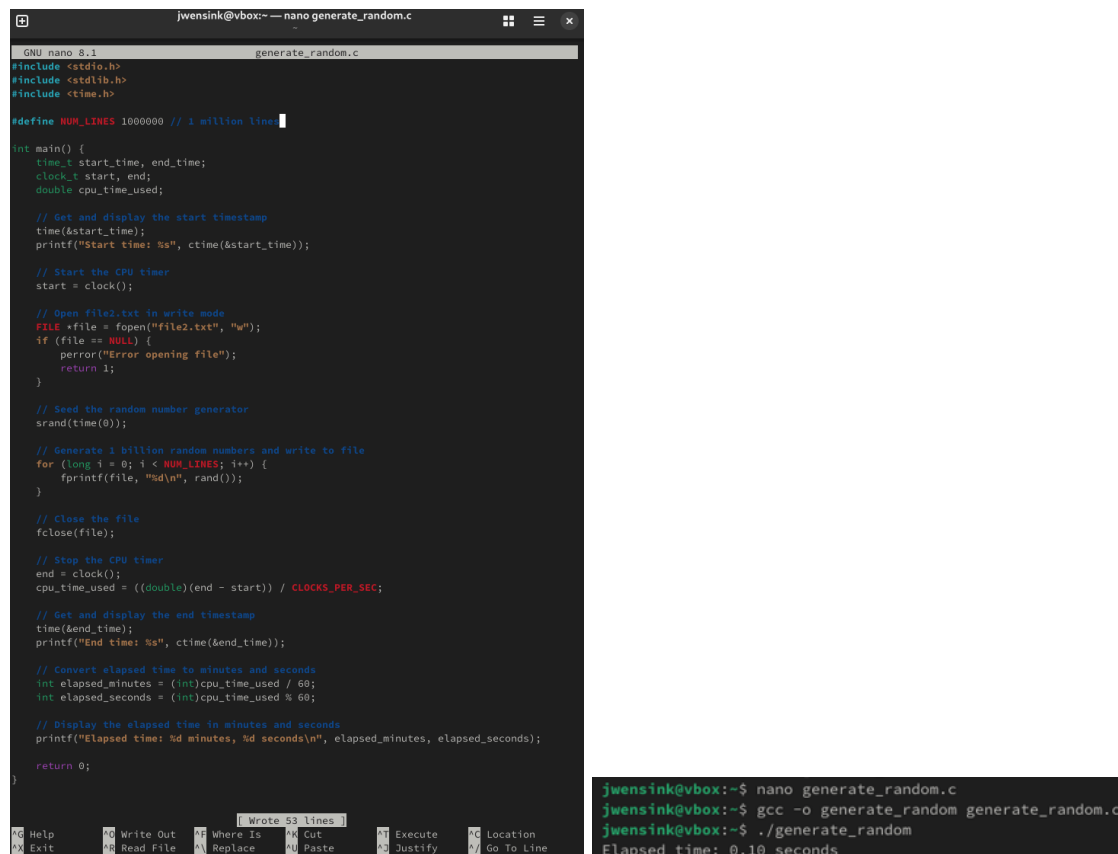
# Generate file1.txt with 1,000,000 random numbers
echo "Generating random numbers..."
for i in $(seq 1 1000000); do
    echo $RANDOM >> file1.txt
done

# Display the system time after the process
echo "End time: $(date)"

# Calculate and display the elapsed time
ELAPSED_TIME=$((SECONDS))
echo "Elapsed time: $((SECONDS / 3600)) hours, $((SECONDS / 60 % 60)) minutes, $((SECONDS % 60)) seconds"
```

```
jwensink@vbox:~$ nano numbers.sh
jwensink@vbox:~$ rm file1.txt
jwensink@vbox:~$ chmod +x numbers.sh
jwensink@vbox:~$ ./numbers.sh
Start time: Sat Nov 30 03:36:59 PM EST 2024
Generating random numbers...
End time: Sat Nov 30 03:38:35 PM EST 2024
Elapsed time: 0 hours, 1 minutes, 36 seconds
```

After the shell script was outputting as expected, I shifted my focus to add the same improvements to my C program. My program from last week, *generate_random.c*, was still available in the persistent storage drive on the Fedora VM, and I updated the *for* loop to generate 1 million numbers for this assignment. To add the clock features, I requested help from a LLM Grimoire (OpenAI, 2024) as I have hardly any experience writing in compiled languages like C with manual memory management. I had to have some features explained to me here, like the library *time* (TutorialsPoint, n.d.-a), the *double* data type, and how to calculate the elapsed CPU time used in terms of seconds. I compiled the code using the Gnu Compiler Collection (GCC) command *gcc -o generate_random generate_random.c* with the *-o* modifier to specify the output file name. I was surprised to see that execution only took 10 seconds.



```
GNU nano 8.1 generate_random.c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define NUM_LINES 1000000 // 1 million lines

int main() {
    time_t start_time, end_time;
    clock_t start, end;
    double cpu_time_used;

    // Get and display the start timestamp
    time(&start_time);
    printf("Start time: %s", ctime(&start_time));

    // Start the CPU timer
    start = clock();

    // Open file2.txt in write mode
    FILE *file = fopen("file2.txt", "w");
    if (file == NULL) {
        perror("Error opening file");
        return 1;
    }

    // Seed the random number generator
    srand(time(0));

    // Generate 1 billion random numbers and write to file
    for (long i = 0; i < NUM_LINES; i++) {
        fprintf(file, "%d\n", rand());
    }

    // Close the file
    fclose(file);

    // Stop the CPU timer
    end = clock();
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;

    // Get and display the end timestamp
    time(&end_time);
    printf("End time: %s", ctime(&end_time));

    // Convert elapsed time to minutes and seconds
    int elapsed_minutes = (int)cpu_time_used / 60;
    int elapsed_seconds = (int)cpu_time_used % 60;

    // Display the elapsed time in minutes and seconds
    printf("Elapsed time: %d minutes, %d seconds\n", elapsed_minutes, elapsed_seconds);

    return 0;
}
```

```
jwensink@vbox:~$ nano generate_random.c
jwensink@vbox:~$ gcc -o generate_random generate_random.c
jwensink@vbox:~$ ./generate_random
Elapsed time: 0.10 seconds
```

This ended up working faster than I had expected, but the early effort did not have the desired timestamps for start and stop. To change this, I added the lines to capture the actual start and stop timestamps, as well as *ctime()*, to convert the timestamps so that they were human-readable. I recompiled the program and was satisfied with the output formatting.

```
jwensink@vbox:~$ nano generate_random.c
jwensink@vbox:~$ gcc -o generate_random generate_random.c
jwensink@vbox:~$ ./generate_random
Start time: Sat Nov 30 15:54:38 2024
End time: Sat Nov 30 15:54:38 2024
Elapsed CPU time: 0.10 seconds
```

The efficiency of the C program is likely due to its being written in a compiled language with minimal overhead. With 1 million lines, I/O disk writes do not seem to be a problem despite having to initialize the VM with SATA speed disk writes instead of the NVMe speeds, which caused the VM to bug and not boot. I wanted to do some additional testing, and to up the ante, I changed the program incrementally to write 10 million, 100 million, and eventually 1 billion lines of random numbers to put the hardware limitations in the spotlight. With 1 billion lines and no optimizations, the program took 102 seconds. Not satisfied with having output read in total seconds, I refactored and recompiled the code again to give output in minutes and seconds.

```
jwensink@vbox:~$ ./generate_random
Start time: Sat Nov 30 15:57:18 2024
End time: Sat Nov 30 15:59:02 2024
Elapsed CPU time: 102.26 seconds
jwensink@vbox:~$ nano generate_random.c
jwensink@vbox:~$ gcc -o generate_random generate_random.c
jwensink@vbox:~$ ./generate_random
Start time: Sat Nov 30 16:02:59 2024
End time: Sat Nov 30 16:04:29 2024
Elapsed time: 1 minutes, 28 seconds
jwensink@vbox:~$ ls -lh file2.txt
-rw-r--r-- 1 jwensink jwensink 9.8G Nov 30 16:04 file2.txt
```

With one billion lines of random numbers, there seems to be room for optimization with multithreading and I/O chunking. The 9.8GB file is enormous and highlights some issues, with the majority of time likely to be spent writing to the file rather than generating the numbers

themselves. At this point, I wanted to start to optimize the program to run on the four physical cores and eight logical processors that were allocated to the VM. I used the *pthread* library (GeeksforGeeks, n.d.) to implement a mutex to ensure safe writing across multiple threads trying to write concurrently. At this point, I'm not ashamed to admit that I was well outside my competency using C and again asked Grimoire (2024) to help me put together some multithreading optimizations. I had gotten through both of this week's chapters in the textbook (Stallings, 2018) and was confident enough with the theory of mutual exclusion, and I felt I was ready to start learning how to implement some of the techniques in a testing environment. I was quite surprised to see that this actually made performance worse, with the program taking 6 minutes and 23 seconds to complete its execution.

```

1 // generate_random_multithread.c
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <time.h>
5 #include <pthread.h>
6
7 #define MAX_THREADS 10000000 // 1 million
8 #define MAX_THREADS_4 // Number of threads
9 #define COUNT_SIZE (MAX_THREADS / MAX_THREADS_4)
10
11 // Global mutex
12 pthread_mutex_t file_mutex;
13
14 // Function executed by each thread
15 void* write_random_numbers(void* arg) {
16     long start = (long)arg; // Starting index for the thread
17     long end = start + COUNT_SIZE;
18
19     srand(time() + start); // Seed random number generator uniquely per thread
20     for (long i = start; i < end; i++) {
21         int random_number = rand();
22
23         // Lock the mutex before writing
24         pthread_mutex_lock(&file_mutex);
25         printf("File: %d\n", random_number);
26         pthread_mutex_unlock(&file_mutex);
27     }
28     return NULL;
29 }
30
31 int main() {
32     pthread_t start_time, end_time;
33     time_t start_time;
34     struct timespec time_start, time_end;
35
36     // Get the start time
37     time(&start_time);
38     printf("Start time: %s", ctime(&start_time));
39
40     // Start the threads
41     start = clock();
42
43     // Create threads to write
44     file = fopen("file.txt", "w");
45     if (!file) {
46         perror("Error opening file");
47         return 1;
48     }
49
50     // Initialize mutex
51     pthread_mutex_init(&file_mutex, NULL);
52
53     // Create and launch threads
54     pthread_t threads[MAX_THREADS];
55     for (long i = 0; i < MAX_THREADS; i++) {
56         pthread_create(&threads[i], NULL, write_random_numbers, (void *) (i + COUNT_SIZE));
57     }
58
59     // Wait for all threads to complete
60     for (long i = 0; i < MAX_THREADS; i++) {
61         pthread_join(threads[i], NULL);
62     }
63
64     // Destroy mutex
65     pthread_mutex_destroy(&file_mutex);
66
67     // Close the file
68     fclose(file);
69
70     // Stop the CPU time
71     end = clock();
72     cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
73
74     // Get the end time
75     time(&end_time);
76     printf("End time: %s", ctime(&end_time));
77
78     // Calculate elapsed time to minutes and seconds
79     int elapsed_minutes = (int)(cpu_time_used / 60);
80     int elapsed_seconds = (int)(cpu_time_used % 60);
81
82     // Display the elapsed time to minutes and seconds
83     printf("Elapsed time: %d minutes, %d seconds\n", elapsed_minutes, elapsed_seconds);
84
85     return 0;
86 }

```

```

jwensink@vbox:~$ nano generate_random_multithread.c
jwensink@vbox:~$ gcc -pthread -o generate_random_multithread generate_random_multithread.c
jwensink@vbox:~$ ./generate_random_multithread
Start time: Sat Nov 30 16:16:08 2024
End time: Sat Nov 30 16:20:12 2024
Elapsed time: 6 minutes, 23 seconds

```

I quickly realized that I was calculating CPU time instead of wall clock time due to the start and end times being 4 minutes and 4 seconds apart. I decided to modify the program's output to specify both CPU time and wall-clock time, and I expected that CPU time would start to approach wall-clock time as the program became more optimized. Still, something was definitely wrong as the program was taking 4 times longer to execute after the multithreading optimizations were included. I suspected that this drop in performance was due to each thread blocking the others in order to write its number to the file, creating significant contention between the four running threads and single I/O disk. To address this, I included some buffer areas for each thread to mitigate any potential bottlenecks caused by the repeated I/O operations. Each thread now gets its own buffer (*buffers[**NUM_THREADS**]*) to store the random numbers, having memory allocated at the beginning of the process and written to disk only after all threads have completed their work. With the average size of a random number being ten digits and a newline character '\n', the memory allocation looks like this.

```
char *buffers[NUM_THREADS];
```

```
[thread_id] = malloc(CHUNK_SIZE * 12);
```

To store the generated numbers in buffers, the *sprintf* function from the *stdio.h* library was used to format the random numbers and append them to the buffer at the *buffer_ptr* location in memory formatted with *%d\n* as an integer on its own new line.

```
buffer_ptr += sprintf(buffer_ptr, "%d\n", rand());
```

This (obviously in hindsight) caused an integer overflow attempting to allocate the massive amount of memory needed for the buffers. Allocating memory with *malloc* using *int* on a 64-bit

system ends up with *int* having a size limit of 32 bits, meaning it can only represent values with values up to approximately 2.1 billion bytes (TutorialsPoint, n.d -b.) When **CHUNK_SIZE * 12** exceeded that limit, it caused an overflow. To prevent this, the data type *size_t* was used to work with the large values. Additionally, having 3GB of memory per thread was an unrealistic expectation for this VM, and the **BUFFER_SIZE** needed to be reduced to 10 million numbers per buffer.

```
jwensink@vbox:~$ gcc -pthread -o generate_random_multithread_4threads generate_random_multithread_4threads.c
generate_random_multithread_4threads.c: In function 'generate_random_numbers':
generate_random_multithread_4threads.c:18:44: warning: integer overflow in expression of type 'int' results in '-1294967296' [-Woverflow]
   18 |     buffers[thread_id] = malloc(CHUNK_SIZE * 12); // Allocate enough space for numbers and '\n'
      |                                ^
generate_random_multithread_4threads.c: In function 'main':
generate_random_multithread_4threads.c:66:42: warning: integer overflow in expression of type 'int' results in '-1294967296' [-Woverflow]
   66 |     fwrite(buffers[i], 1, CHUNK_SIZE * 12, file);
      |                                ^
generate_random_multithread_4threads.c: In function 'generate_random_numbers':
generate_random_multithread_4threads.c:18:26: warning: argument 1 value '18446744072414584320' exceeds maximum object size 9223372036854775807 [-Walloc-size-larger-than=]
   18 |     buffers[thread_id] = malloc(CHUNK_SIZE * 12); // Allocate enough space for numbers and '\n'
      |                                ^
In file included from generate_random_multithread_4threads.c:2:
/usr/include/stdlib.h:672:14: note: in a call to allocation function 'malloc' declared here
  672 | extern void *malloc (size_t __size) __THROW __attribute_malloc__
```

These fixes resolved the overflow issues, and I was able to compile and run the program with a much more reasonable execution time to create a file with a billion random numbers.

```
jwensink@vbox:~$ ./generate_random_multithread_4threads
Start time (wall-clock): Sat Nov 30 16:45:55 2024
End time (wall-clock): Sat Nov 30 16:47:32 2024
Elapsed time (wall-clock): 1 minutes, 37 seconds
Elapsed time (CPU): 4 minutes, 7 seconds (247.45 seconds total)
```

Still, I thought I could perhaps do better multithreading by using my four physical cores' hyperthreading capabilities by increasing the number of threads to eight. I ran *lscpu* to make sure that I remembered the CPU configuration correctly, and this supported the fact that there were eight logical processors available to run threads.

```
jwensink@vbox:~$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          48 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 8
On-line CPU(s) list:    0-7
Vendor ID:              AuthenticAMD
Model name:             AMD Ryzen 7 5800X 8-Core Processor
CPU family:            25
Model:                 33
Thread(s) per core:     1
Core(s) per socket:     8
```

I increased the number of active threads to 8 and noticed an immediate decrease in performance.

```
jwensink@vbox:~$ ./generate_random_multithread_8threads
Start time (wall-clock): Sat Nov 30 16:51:12 2024
End time (wall-clock):  Sat Nov 30 16:53:05 2024
Elapsed time (wall-clock): 1 minutes, 53 seconds
Elapsed time (CPU): 7 minutes, 18 seconds (438.73 seconds total)
```

This was likely due to the 4 physical cores needing to split their resources to handle two threads per core, introducing additional context-switching overhead as the threads are now sharing the same resources in caches and execution units. Hyperthreading can be beneficial for tasks that include a lot of waiting for I/O operations. Still, in this case, it provided a performance decrease for the CPU-intensive task of generating random numbers. Regardless of how many threads are run, the main issue that I believe is preventing efficient execution must be the disk write bottleneck.

The second optimization strategy I pursued for this assignment was to have each thread write to a separate, temporary binary file *file_thread<N>.bin* (OpenAI, 2024.) This should help

to avoid the mutex contention for trying to write to a single file, with the temporary files merged at the end into the final output file2.bin. This not only led to reduced I/O contention but also resulted in a smaller file size with minimal overhead needed for the final merge. This addition led to the quickest execution time yet, just 22 seconds of wall-clock time and 49 seconds of CPU time to write 1 billion lines of binary integers.

```
jwensink@vbox:~$ ./generate_random_multithread_io_optimized
Start time (wall-clock): Sat Nov 30 16:59:59 2024
End time (wall-clock): Sat Nov 30 17:00:21 2024
Elapsed time (wall-clock): 0 minutes, 22 seconds
Elapsed time (CPU): 0 minutes, 49 seconds (49.08 seconds total)
```

While quick, saving a .bin file does not meet the assignment requirements, which require the file to be saved as a .txt file. Hoping to keep the optimizations just gained, I tried to convert the file at the end, keeping the temporary files in binary format and converting the binary data into textual format when merging the temporary files into the final output (OpenAI, 2024.)

This caused another decrease in performance, causing the program to take 1 minute 49 seconds of wall-clock time and 2 minutes 18 seconds of CPU time, worse than before introducing the .bin I/O optimization in the first place.

```
jwensink@vbox:~$ ./random_numbers_bin_to_text
Start time (wall-clock): Sat Nov 30 21:45:22 2024
End time (wall-clock): Sat Nov 30 21:47:11 2024
Elapsed time (wall-clock): 1 minutes, 49 seconds
Elapsed time (CPU): 2 minutes, 18 seconds (138.68 seconds total)
jwensink@vbox:~$
```

I found this drop in performance to be unacceptable, but I didn't want to abandon the .bin to .txt approach just yet as it avoids the overhead of formatting numbers into strings during the initial

Week 3 Portfolio Milestone in C

10

generation phase. My next thought to keep these performance gains while still outputting a .txt file was to test if it would be beneficial to parallelize the merge phase itself. Grimoire (2024) recommended a solution to implement this plan to keep the binary intermediary and parallelize the final merge into a .txt file.

```
pthread_t merger_threads[NUM_THREADS];
pthread_mutex_t file_mutex = PTHREAD_MUTEX_INITIALIZER;
merge_data_t merge_data[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++) {
    merge_data[i].thread_id = i;
    merge_data[i].output_file = output_file;
    merge_data[i].file_mutex = &file_mutex;
    pthread_create(&merger_threads[i], NULL, merge_to_text, &merge_data[i]);
}

for (int i = 0; i < NUM_THREADS; i++) {
    pthread_join(merger_threads[i], NULL);
}
```

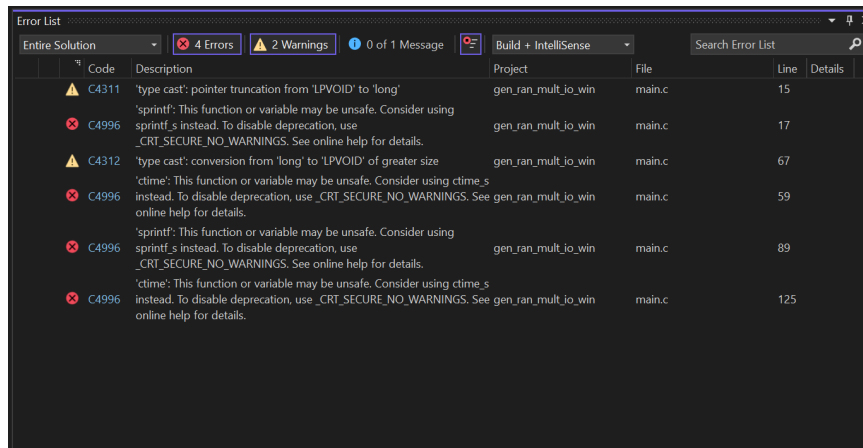
```
jwensink@vbox:~$ nano ran_gen_parallel_merge.c
jwensink@vbox:~$ gcc -o ran_gen_parallel_merge ran_gen_parallel_merge.c -lpthread
jwensink@vbox:~$ ./ran_gen_parallel_merge
Start time (wall-clock): Sun Dec 1 09:53:50 2024
End time (wall-clock): Sun Dec 1 09:54:40 2024
Elapsed time (wall-clock): 0 minutes, 50 seconds
Elapsed time (CPU): 2 minutes, 0 seconds (120.39 seconds total)
jwensink@vbox:~$ ls | grep file2.txt
file2.txt
jwensink@vbox:~$ wc -l file2.txt
1000000000 file2.txt
```

This optimization seems to have worked favorably, with the execution taking just 50 seconds of wall-clock time and 120.39 seconds of CPU time. The program effectively minimizes the computational overhead during the initial generation phase by deferring the expensive operation of formatting numbers to text. Instead of handling the resource-intensive task of converting integers to strings and then immediately writing them to a .txt file, the optimization makes use of fast binary writes to store the raw data more efficiently. The binary serves as a compact, intermediate storage, reducing the I/O burden substantially compared to directly writing a large .txt file. The subsequent parallelized merge step further exploits this advantage by distributing the text conversion workload across multiple threads. By batching the text writes into memory and committing them to the final .txt file in larger chunks, the program reduces the frequency of disk I/O operations and avoids the constant flushing that would need to occur with line-by-line text output.

I was curious about how this code would run on my host Windows environment without the overhead of a VM and all local resources dedicated to execution. I started up Microsoft Visual Studio 2022 and started a new project in Desktop Development with C++ module. I started a new project and began working on a new item named *main.c* to ensure that Visual Studio was using the C11 language standard and not C++. Once again, Grimoire (2024) guided me in fixing any outstanding compatibility issues. Since Windows does not natively support POSIX threads, I had to replace any references to *pthread.h* (*pthread_create*, *pthread_join*) with Windows Threads API *windows.h* (*CreateThread*, *WaitForSingleObject*.) For this to work the *DWORD WINAPI* function needed to be invoked at the beginning of the program. Other issues that came up when switching from Linux to Windows Visual Studio, such as needing to change *long* to *uintptr_t* for the *thread_id* to match pointer sizes as *long* might not have been large enough to store the pointer value on a 64-bit system. Similarly, *(LPVOID)i* needed to be updated to *(LPVOID)(winptr_t)i* due to the same issue potentially causing a mismatch on a 64-bit system. There was an error where Visual Studio had marked *sprintf* as unsafe due to not performing a bounds check, potentially leading to a buffer overflow. The solution was to replace *sprintf* with *sprintf_s* and specifying the buffer size. Another unsafe function was *ctime*, as it uses a static internal buffer that could be overwritten by other functions. The fix was again to replace *ctime* with *ctime_s* and allocate an appropriate buffer size. In this case, 26 bytes was sufficient.

Week 3 Portfolio Milestone in C

12



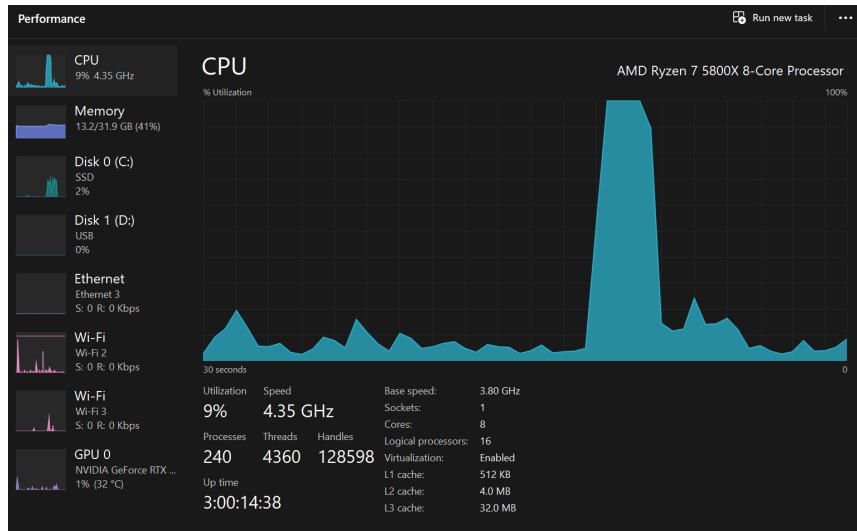
With 4 threads running on eight physical cores and full access to the NVMe disk write speeds, the program took only 8.91 seconds of CPU time and 9 seconds of wall-clock time to write a billion lines of random numbers. The similarity between the CPU and wall-clock times suggests to me that the program executed with near-perfect CPU thread synchronization. The low execution times really showcase the raw speed of NVMe writes, suggesting that the I/O operations are so fast that threads aren't bottlenecked for any significant period of time waiting for the disk. Still, I think we can do even better by increasing the threads to eight to match the eight physical cores available to do work. By doubling the threads from four to eight, the program executed in 7 seconds of CPU time and 7 seconds of wall clock time. For the final optimization, I had the program run with 16 threads to make use of the hyperthreading logical processors and expected to see decreased performance due to resource contention. However, when I ran the test, I noticed a modest increase in performance for the fastest time yet, at 5.65 seconds CPU time and 6 seconds wall-clock time.

Week 3 Portfolio Milestone in C

13

```
Microsoft Visual Studio Debu: x + v
Start time (wall-clock): Sat Nov 30 18:35:32 2024
End time (wall-clock): Sat Nov 30 18:35:38 2024
Elapsed time (wall-clock): 0 minutes, 6 seconds
Elapsed time (CPU): 0 minutes, 5 seconds (5.65 seconds total)

C:\Users\jwens\OneDrive\Desktop\CSUGlobal\CSC507\gen_ran_mult_io_win\x64\Debug\gen_ran_mult_io_win.exe (process 11128) exited with c
ode 0 (0x0).
Press any key to close this window . . .
```



After the Windows testing, I was happy with the optimizations. I went back to Fedora and changed the code back to meet the project requirements of creating just one million lines instead of 1 billion. This workload executed in 1.06 seconds of CPU time and 2 seconds of wall-clock time. I saved the file on my host PC for submission as `Portfolio_Milestone_Week_3.c`.

```
jwensink@vbox:~$ nano ran_gen_parallel_merge.c
jwensink@vbox:~$ gcc -o ran_gen_parallel_merge ran_gen_parallel_merge.c -lpthread
jwensink@vbox:~$ ./ran_gen_parallel_merge
Start time (wall-clock): Sun Dec 1 13:18:58 2024
End time (wall-clock): Sun Dec 1 13:19:00 2024
Elapsed time (wall-clock): 0 minutes, 2 seconds
Elapsed time (CPU): 0 minutes, 1 seconds (1.06 seconds total)
```

If my system had double its current CPU power, I would expect a modest increase in performance. I think that since my disk usage didn't max out, nor did my memory, another 16

logical processors would likely allow more threads to run concurrently. With the highly parallelizable random number generation task, I would expect this stage to benefit proportionally. The binary-to-text stage also benefits from parallelism but also involves I/O disk writes. I would expect to see linear performance increases up to the disk's max throughput and then expect quickly diminishing returns once this throughput is reached. It seems like all 16 logical processors must be running efficiently with minimal overhead. With only about ~0.5 seconds difference between CPU time and wall-clock time, I think that the parallelism, synchronization, and chunk size must be approaching their optimal configuration. I think the more interesting question would be, would doubling the existing cores' clock speed give an even better performance boost? Unlike adding more cores, increasing clock speed wouldn't introduce additional synchronization overhead, so every thread would be proportionally faster with faster mutex locks in the merge step. This would make doubling the clock speed a more compelling option than doubling the number of processors (without considering the cost) for this particular task, up to the point where I/O or memory becomes the bottleneck. In the article *Speedup Stacks: Identifying Scaling Bottlenecks in Multithreaded Applications* (Eyerman, Du Bois, & Eeckhout, 2012), the authors discuss how various factors impact the scalability of multithreaded applications as hardware resources are increased. Key bottlenecks such as synchronization overhead, resource contention, and load imbalance are shown to limit the effectiveness of adding more cores to a system. This article supports my theory that while doubling CPU cores or their clock speeds can theoretically lead to linear performance improvements, other system bottlenecks like I/O and thread synchronization would likely constrain the actual gains. For this task, I think it is likely that the random number generation stage would scale well with additional

processing resources, but the binary-to-text stage is likely to face immediate diminishing returns as disk throughput becomes the bottleneck. Ultimately, the article's findings show the importance of understanding the specific characteristics of a workload when deciding how to increase specific system resources.

Conclusion

This project provided a terrific opportunity to get acquainted with the C programming language. Before this class, I had no experience with compiled languages, and it was an excellent opportunity to step well outside my comfort zone. Through hands-on experimentation and iterative refinement, I not only gained insight into the structure and syntax of C but also explored more advanced concepts like memory management, multithreading, and synchronization. Having an LLM to help guide me through the more challenging aspects of the project and explain how to code abstract concepts at a low level allowed me to approach complex ideas and implement solutions that I otherwise would not have had the skill or experience to seek out from more traditional sources like StackOverflow or GitHub discussion boards. Having the ability to ask specific questions and receive immediate, tailored explanations represents one of the most transformative educational tools that has ever been created. I expect that LLMs will revolutionize the way programming is taught and learned, making complex concepts more accessible and empowering students to approach challenges they would otherwise not have thought possible. By going beyond finding prewritten code or partial solutions and truly engaging with the process step by step, I developed a deeper appreciation for the flexibility, power, and sheer speed of C programming. The process of optimizing system resources helped me gain a solid foundation to

continue exploring system-level programming and has improved my confidence in taking on future challenges.

The progression from a simple shell script to a fully optimized C program in both Linux and Windows environments was a journey that involved several stages of development and incremental refinement. Starting with a simple shell script written in Bash to generate random numbers and then write them to a file provided a baseline for functionality and performance measurement. Next, the transition to a single-threaded C program brought substantial performance improvements by using the power of compiled code for faster execution. Using the *time.h* library as a measuring stick for performance allowed me to incorporate *time()* to track wall-clock seconds and *clock()* to track CPU time. This delivered details into the efficiency of multithreaded operations and provided a clear benchmark for comparing performance improvements and setbacks. After it became clear that generating one million numbers would be an insufficient workload to showcase the performance limitations of the system, the workload was incrementally scaled up to more demanding levels. This scaling exposed bottlenecks, particularly in I/O writes, as the sheer size of the output magnified any inefficiencies in resource management. To address these issues, multithreading was introduced using the *pthread* library. Initially, this led to contention and inefficiencies due to mutex locks and limited I/O throughput. The program was then refined to use binary files instead of textual writes, significantly reducing contention and improving performance. The project requirements, however, specified to output a .txt file, which presented an interesting challenge: how to retain the performance benefits of binary writes while still meeting the requirements for textual output. To address this, the program was modified to use binary files as the intermediate storage, deferring the conversion of binary

data until the final merge phase. This introduced additional complexity in managing synchronization and buffering across multiple threads. By postponing the computationally expensive task of converting binary into text until the final merge phase significantly reduced execution times compared to a direct-to-text output strategy while still delivering the required .txt file as the final output. After meeting the project requirements, I was still curious about how the program would perform running directly on the host OS with exclusive use of the CPU cores and the machine's full NVMe disk speeds. After configuring the program to use Windows-supported libraries and system APIs, the improvements in performance were remarkable. Unlike the VM, which was limited by slower SATA disk speeds and limited processing cores, the host OS executed with exceptional efficiency. Testing the program using 16 active threads, one for each logical processor, allowed me to utilize my CPU's support for hyperthreading, resulting in the peak performance of one billion random numbers written to text in 5.65 seconds of CPU time and 6 seconds of wall-clock time. This suggested that the program was running efficiently and highlighted the differences between running code in a VM versus native hardware.

This journey was one of the most interesting school projects I've worked on so far. I look forward to further refining this program as we learn more about operating systems and the C programming language. I gained valuable experience with both this week, and feel that I have a great start on my final Portfolio Project.

References

Eyerman, S., Du Bois, K., & Eeckhout, L. (2012). Speedup stacks: Identifying scaling bottlenecks in multi-threaded applications. Retrieved December 1, 2024, from https://users.elis.ugent.be/~leeckhou/papers/ispass12_2.pdf

GeeksforGeeks. (n.d.). Multithreading in C. Retrieved November 30, 2024, from <https://www.geeksforgeeks.org/multithreading-in-c/>

Grimoire. (2024). Conversation about the C programming language. OpenAI. Retrieved November 30, 2024, from <https://chat.openai.com>

Stallings, W. (2018). Operating systems: Internals and design principles (9th ed., Global ed.). Pearson. Retrieved November 30, 2024, from https://www.youseficlass.ir/wp-content/uploads/2023/07/William_Stallings_Operating_Systems_Internals_and_Design_Principles_watermark.pdf

TutorialsPoint. (n.d.-a). C Library - <time.h>. Retrieved November 30, 2024, from https://www.tutorialspoint.com/c_standard_library/time_h.htm

TutorialsPoint. (n.d.-b). C Library - <limits.h>. Retrieved November 30, 2024, from https://www.tutorialspoint.com/c_standard_library/limits_h.htm